

שבוע אחד עשר על עמוד אחד

שבוע: 11

המשפט אחד של השבוע

חוט אחד. `await` לא מוסיף חוט – הוא מחזיר אותו.

השכבה החמישית

שבוע 9 נתן שלוש שכבות ושלושה חוקים, שבוע 10 הוסיף רביעית. השבוע מוסיף חמישית, ואף אחת מהקודמות לא זזה.

הקובץ	התפקיד	מה אסור לו
<code>js/state.js</code>	האובייקט וכל הגזירות	לגעת ב-DOM
<code>js/render.js</code>	<code>state</code> נכנס, מסך יוצא	לשנות <code>state</code> · לרשום מאזינים · <code>async</code>
<code>js/events.js</code>	פעולת משתמש הופכת לשינוי <code>state</code>	לצייר · <code>fetch</code>
<code>js/storage.js</code>	האובייקט הופך למחרוזת, ובחזרה	לגעת ב-DOM · לזרוק
<code>js/api.js</code>	בקשה הופכת לנתונים, או לשגיאה	לגעת ב-DOM · <code>setState</code> · להחזיר חצי-תשובה
<code>js/app.js</code>	לחבר את החמישה, פעם אחת	לגדול. השבוע הוא בכלל לא משתנה

המילה `fetch` מופיעה בקובץ אחד בלבד. אותו דבר לגבי כתובת ה-API. חפש את שתיהן ב- `render.js` וב- `events.js` כשסיימת.

שלוש עובדות על `await` שכל השבוע עומד עליהן

#	העובדה	מה זה עולה
1	העולם זז בזמן שאתה ממתיין	כלום ממה שקראת לפני ה- <code>await</code> עדיין מובטח אחריו
2	תשובות לא מגיעות בסדר שבו שאלת	הבקשה האיטית היא לרוב הישנה, והיא נוחתת אחרונה
3	ביטול מגיע כדחייה	<code>catch</code> נאיבי מתייחס לביטול שלך ככישלון

מה `fetch` לא עושה

מה שנדמה	מה שבאמת
זורק על 404 ועל 500	לא. ההבטחה מתקיימת, ו- <code>res.ok</code> הוא <code>false</code>
יש לו פסק זמן	אין. ומעולם לא היה. בקשה יכולה להיתקע לנצח

מה שנדמה	מה שבאמת
התשובה היא הנתונים	לא . <code>res.json()</code> הוא <code>await</code> שני ודבר שני שיכול להיכשל
<code>try/catch</code> מכסה	רק את המקרה שבו הבקשה לא הושלמה . זה אחד מתוך חמישה

`fetch` נדחה במקרה אחד בלבד: **הבקשה לא הושלמה**. DNS, אין נתיב, CORS, ביטול. זהו.

```
const res = await fetch(url, { signal });
if (!res.ok) throw new ApiError('status', '...', `HTTP ${res.status}`);
let data;
try {
  data = await res.json();
} catch {
  throw new ApiError('shape', '...', '...');
}
return normalise(data);
```

חמשת המצבים

`#api-panel` נושא `data-state`, ותמיד **בדיוק אחד** מהם:

המצב	מתי	ומה זה לא
<code>idle</code>	עוד לא חיפשו כלום	<code>empty</code> . אף אחד לא שאל
<code>loading</code>	בקשה פתוחה	ולא אחרי ה- <code>await</code> — שם זה כבר לא נכון
<code>error</code>	נכשלה, ויש משפט	משפט וגם דרך קדימה , לא "משהו השתבש"
<code>empty</code>	הצליחה, ולא מצאה כלום	לא שגיאה. השרת ענה, והתשובה היא "אין"
<code>results</code>	הצליחה, ומצאה	

שדה אחד עם חמישה ערכים, לא חמישה בוליאנים. חמישה בוליאנים הם 32 צירופים שרק חמישה מהם אומרים משהו — ואת השאר מייצר `isEmpty = false` אחד ששכחו.

```
if (status === 'loading') return 'loading';
if (status === 'error') return 'error';
if (status === 'idle') return 'idle';
return results.length === 0 ? 'empty' : 'results';
```

`empty` נגזר. `loading` נשמר. "האם חזר משהו" היא שאלה על `results`. "האם יש בקשה פתוחה" לא נגזרת משום דבר.

```
let controller = null;
let seq = 0;

async function run(query) {
  controller?.abort(); // 1 – the previous one is not wanted
  controller = new AbortController();
  const ticket = ++seq; // 2 – a ticket per search

  setState({ status: 'loading', error: null }); // 3 – BEFORE the await

  const results = await search(query, { signal: controller.signal });

  if (ticket !== seq) return; // 4 – a newer search has started
  setState({ status: 'done', results });
}
```

השורה	מה היא עוצרת	ולמה שתיהן
<code>abort()</code>	את הרשת	הבקשה נגמרת ולא ממשיכה לרוץ
<code>ticket !== seq</code>	את הכתיבה	תשובה שכבר הגיעה לפני שהביטול נחת עדיין מריצה את ההמשך שלה

ולפני שתי אלה – **השהיה**: `clearTimeout` ואז `setTimeout(..., 300)`. מילה בת שש אותיות היא בקשה אחת ולא שש.

AbortError הוא לא שגיאה:

```
if (error.name === 'AbortError') return;
```

לפי שם ולא לפי מחלקה – `TimeoutError` הוא גם `DOMException`, והוא כן שגיאה.

CORS בשלוש שורות

הבקשה יצאה. השרת ענה. הדפדפן סירב למסור לך את התשובה, כי היא לא נשאה כותרת שאומרת שמותר לך לקרוא אותה.

לכן השגיאה היא `TypeError: Failed to fetch` בלי שום פרט – **אותו אובייקט בדיוק** שככל מנותק מייצר. הסיבה מודפסת בקונסולה על ידי הדפדפן, ואי אפשר לקרוא אותה מ-JavaScript.

אפשר	אי אפשר
לבחור API ששולח את הכותרת	כל דבר ב-JavaScript שלך
שרת משלך באמצע (אין לו דפדפן)	<code>mode: 'no-cors'</code> – נותן תשובה אטומה בלי גוף
לבקש מבעל ה-API להוסיף	תוסף שמכבה את הבדיקה – מתקן רק את הלפטופ שלך
	<code>try/catch</code> – תופס את השגיאה, לא מביא נתונים

```
npm run slides:serve
# http://localhost:5173/weeks/week-11-async-fetch/demos/01-blocking/index.html
```

#	התיקיה	הרעיון האחד
01	01-blocking	אותן שתי שניות, שלוש דרכים. אחת מהן עוצרת את העמוד
02	02-callbacks-to-await	אותה עבודה, שלוש צורות, וכמה מקומות צריך לטפל בכישלון
03	03-promise-states	דחייה שאיש לא מקשיב לה, ומתי הדפדפן מוותר עליה
04	04-microtasks	הסדר האמיתי, ו- <code>setTimeout(fn, 0)</code> שממתין 65 מילישניות
05	05-all-vs-allsettled	לולאה מול <code>all</code> מול <code>allSettled</code> מול <code>race</code> , עם שעון
06	06-ok-is-not-caught	שש תשובות, וחמש מהן לא נדחות
07	07-cors	אותו קוד, שני שרתים, שתי תוצאות
08	08-abort-timeout-retry	כמה זמן ממתינים בלי תקרה, ומה ניסיון חוזר עולה
09	09-stale-search	תיבת החיפוש שעונה על השאלה הלא נכונה, וארבע שורות
10	10-four-states	חמישה מצבים מול ארבעה בוליאנים, ושלוש סתירות

כשהרשת לא משתפת פעולה

```
node mock-api/server.mjs
```

API קטן שרץ אצלך, בלי התקנות. יש בו `?fail=404`, `?fail=500`, `?fail=html`, `?delay=8000` ו- `?hang=1` – כלומר כל התקלות שצריך לבדוק, בפקודה. פרטים ב- `mock-api/README-he.md`.

שבע הטעויות של השבוע

1. שכחת `await`. `items.length` הוא `undefined`, כי `items` הוא `Promise`.
2. `await` בתוך לולאה לחמש בקשות שאינן תלויות זו בזו. פי חמישה זמן, בחינם.
3. הנחת ש- `fetch` זורק על 404. הוא לא. `res.ok` הוא הבדיקה.
4. אין מצב שגיאה בממשק. המשתמש רואה מסך שלא זז ולא יודע למה.
5. בקשה בכל הקשה. שש אותיות, שש בקשות, וחמש מהן איש לא רצה.
6. דחייה שאיש לא תפס. `unhandledrejection` בקונסולה, ומסך שלא מגיב.
7. "CORS זה באג בקוד שלי". לא. זה הדפדפן, והוא עושה את זה בכוונה.

`js/api.js` הוא הקובץ שבו הנחה שגויה על מבנה התשובה הופכת לקריסה בזמן ריצה — כי מה שחוזר משם נכתב על ידי מישהו אחר. **בשבוע 12 המהדר יתפוס את זה לפני שתריץ.**